BME513 - Inteligencia Artificial en Salud

Tree-based methods and ensembles

Alejandro Veloz

Learning goals

- Understand why ensembles often generalize better than a single model.
- Build a decision tree from recursive, greedy splits.
- Compare classification and regression trees.
- Explain bagging, random forests, boosting, and AdaBoost.
- Connect ensemble diversity with error correlation and mixture of experts.

Ensembles

Suppose we have several imperfect predictors:

$$f_1(x), f_2(x), \dots, f_M(x).$$

An ensemble combines them:

$$F(x) = \sum_{m=1}^{M} \alpha_m f_m(x).$$

The goal is not to make every model perfect.

The goal is to make their mistakes useful when combined.

Ensemble intuition

For classification:

$$F(x) = \text{mode}\{f_1(x), \dots, f_M(x)\}.$$

For regression:

$$F(x) = \frac{1}{M} \sum_{m=1}^{M} f_m(x).$$

Averaging reduces variance.

Voting can cancel individual mistakes.

Three ways to combine models:

- Average numeric predictions.
- Vote for class labels.
- Learn weights when some models are more reliable.

The gain depends on both:

- Individual accuracy.
- Diversity of errors.

Diversity

Ensembles need base models that are:

- Accurate enough on their own.
- Different enough from each other.
- Combined by voting, averaging, or learned weights.

This is sometimes discussed as negative correlation learning: encourage models to avoid making the same errors.

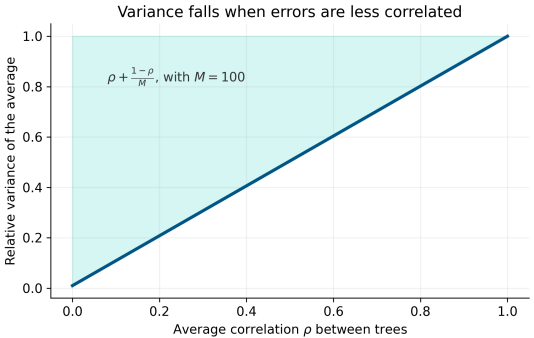
Negative correlation and diversity

If base models have similar variance and average error correlation ρ:

$$\frac{\mathrm{Var}(\bar{f})}{\sigma^2} \approx \rho + \frac{1-\rho}{M}.$$

As M grows, the limit is controlled by ρ.

More models help less when their errors are highly correlated.



Negative correlation learning

In explicit negative correlation learning, each model is trained with a penalty:

$$\mathcal{L}_m = \sum_{i=1}^{n} \left(y_i - f_m(x_i)\right)^2 + \lambda \sum_{i=1}^{n} \left(f_m(x_i) - \bar{f}(x_i)\right) \times \sum_{\ell \neq m} \left(f_\ell(x_i) - \bar{f}(x_i)\right).$$

where

$$\bar{f}(x_i) = \frac{1}{M} \sum_{m=1}^{M} f_m(x_i).$$

The penalty discourages learners from making the same deviations.

Three ensemble families

Method	How diversity appears	Combination
Bagging	different bootstrap datasets	average / majority vote
Random Forest	bootstrap + random features	average / majority vote
Boosting	sequential focus on mistakes	weighted sum
Mixture of experts	different regions or regimes	learned gate

Trees are a natural base learner: high variance, fast, and interpretable.

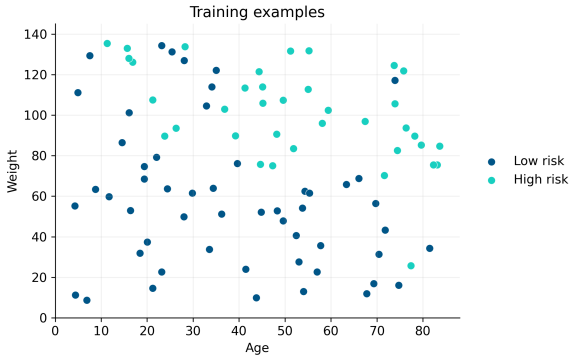
Example

We want to predict patient risk using:

- Age
- Weight
- A binary label.

High risk and low risk form regions in feature space.

A tree will approximate these regions using axis-aligned rules.



Decision rules

A tree is a collection of rules:

$$\text{IF Age} > 45 \text{ AND Weight} > 70 \quad \text{THEN High risk.}$$

Each root-to-leaf path is one rule.

Learning a tree means learning the rules from data.

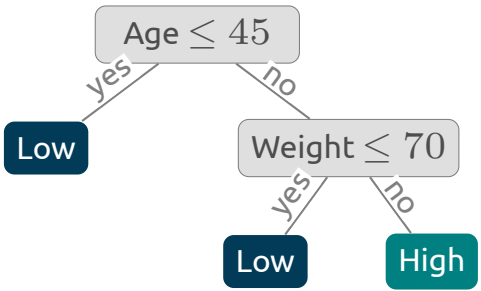
What is a decision tree?

A decision tree is a hierarchical predictive model.

- Internal nodes ask questions.
- Branches are answers.
- Leaves make predictions.

For tabular data, common questions are:

$$x_j \leq t?$$



Recursive partitioning

Tree learning is usually top-down recursive partitioning.

1. Choose the best split at the current node.
2. Divide the data into two child nodes.
3. Repeat the same procedure in each child.
4. Stop when a criterion is met.

The algorithm is greedy: it optimizes the next split, not the whole tree.

Step 1: evaluate candidate splits

```
def best_split(X_node, y_node):
    best = None
    for j in range(X_node.shape[1]):
        for t in candidate_thresholds(X_node[:, j]):
            left = X_node[:, j] <= t
            right = ~left
            gain = impurity(y_node) - weighted_impurity(y_node[left], y_node[right])
            if best is None or gain > best["gain"]:
                best = {"feature": j, "threshold": t, "gain": gain}
    return best
```

Implementation

Real tree implementations avoid the naive cost of trying every split from scratch.

Common optimizations:

- Sort each feature once, then scan candidate thresholds in order.
- Update class counts incrementally while moving the threshold.
- Ignore invalid splits early using `min_samples_leaf` or missing-value rules.
- Use feature subsampling and parallelism across nodes or trees.
- Use histogram/binning approximations in large-scale gradient boosting.

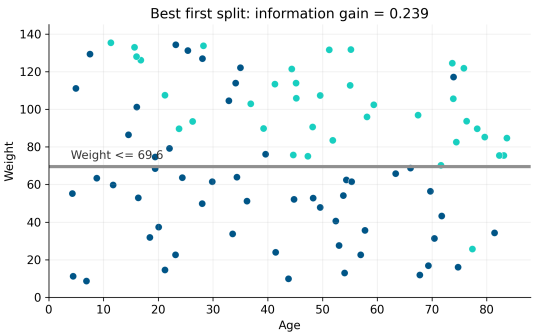
Step 1: the first split

At the root node, the algorithm asks:

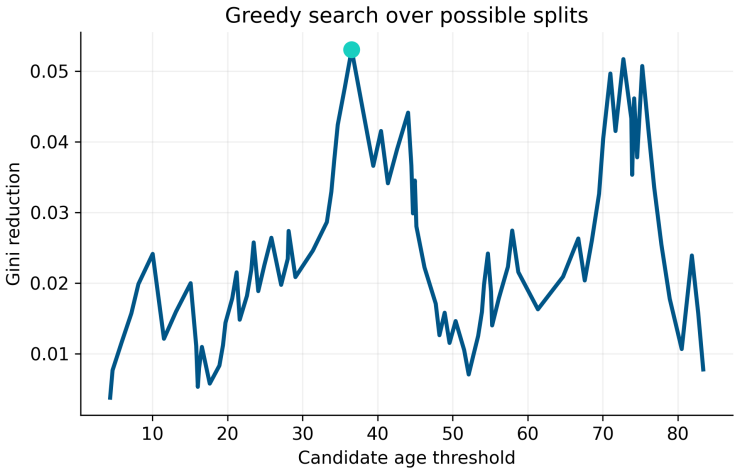
Which feature and threshold most reduce impurity?

For this dataset, the best first split is a vertical cut on age.

The split creates two child nodes with different class mixtures.



Searching over thresholds



Impurity

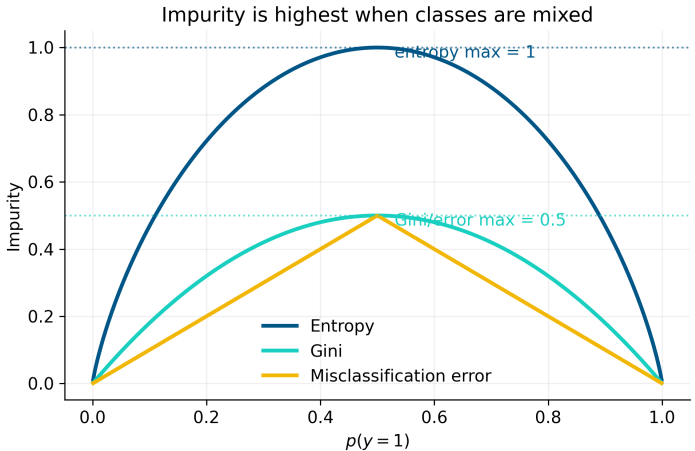
A node is pure when almost all examples have the same label.

For class proportions $p_1, \ldots, p_K$:

$$\text{Entropy} = -\sum_{k=1}^{K} p_k \log_2 p_k, \qquad \text{Gini} = 1 - \sum_{k=1}^{K} p_k^2.$$

Both are low for pure nodes and high for mixed nodes.

Impurity curves



Information gain

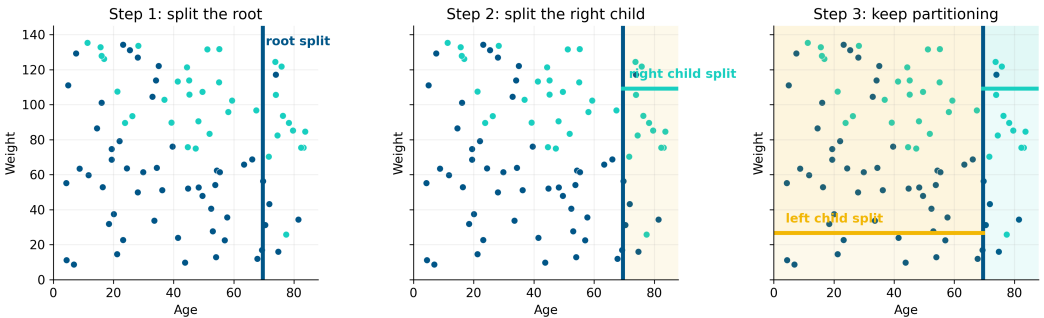
For a candidate split into left and right children:

$$IG = J(P) - \left(\frac{n_L}{n_P} J(L) + \frac{n_R}{n_P} J(R) \right),$$

where $J(\cdot)$ is an impurity measure.

Good splits produce purer child nodes.

Step 2: recurse on each child



Step 3: stop growing

Common stopping criteria:

- Maximum depth.
- Minimum samples per leaf.
- Minimum impurity decrease.
- Pure node.
- No valid split remains.

Stopping controls the bias-variance tradeoff:

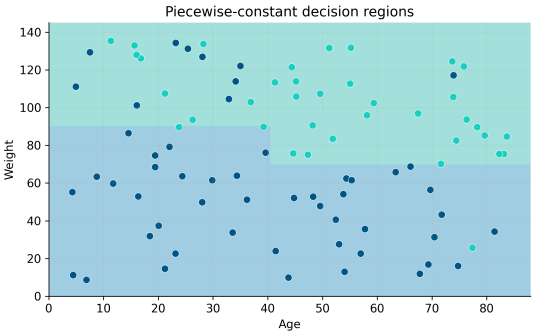
$$\text{shallow tree} \Rightarrow \text{high bias}, \qquad \text{deep tree} \Rightarrow \text{high variance}.$$

Final tree regions

The learned tree predicts a constant class inside each leaf.

This gives:

- Interpretable rules.
- Non-linear decision boundaries.
- Abrupt changes at split thresholds.



Trees are flexible but unstable.

Classification vs regression trees

Task	Leaf prediction	Split criterion
Classification	majority class / class probabilities	Gini or entropy reduction
Regression	mean response in the region	MSE reduction

Both use the same recursive partitioning idea.

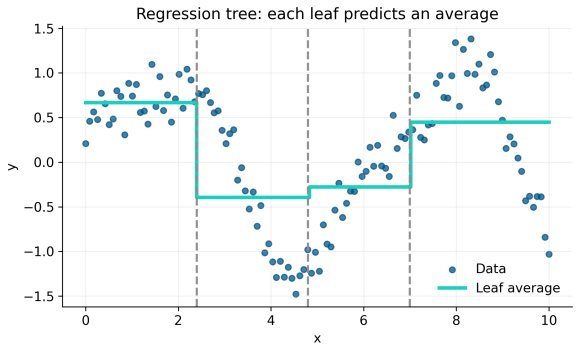
Regression trees

For a regression leaf R_m, the prediction is:

$$\hat{y}_{R_m} = \frac{1}{|R_m|} \sum_{i:x_i \in R_m} y_i.$$

The split is chosen to reduce:

$$\sum_{i:x_i \in R_L} (y_i - \bar{y}_L)^2 + \sum_{i:x_i \in R_R} (y_i - \bar{y}_R)^2.$$



Overfitting and pruning

Deep trees can memorize training noise.

Two common controls:

- Pre-pruning: stop early using depth, leaf size, or impurity decrease;
- Post-pruning: grow a large tree, then remove branches using validation.

The best tree is rarely the largest tree.

Why a single tree is not enough

Decision trees have high variance:

- Small changes in data can change the root split.
- Early split changes affect all descendants.
- Deep trees fit local noise.

This makes trees excellent candidates for ensembles.

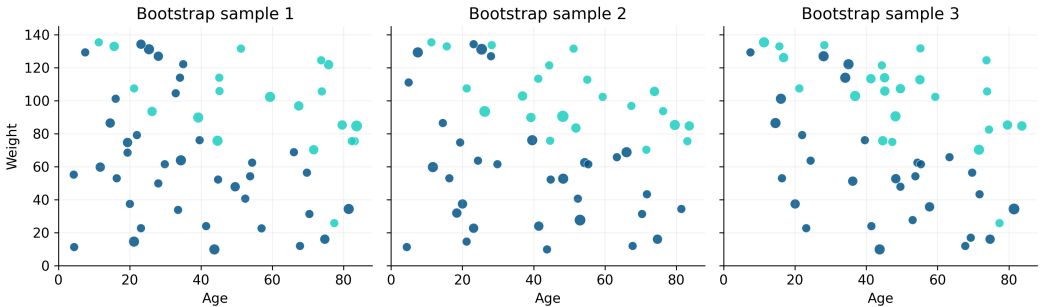
Bagging

Bagging = **bootstrap aggregating**.

1. Draw bootstrap samples from the training set.
2. Train one tree on each sample.
3. Aggregate predictions by voting or averaging.

$$F(x) = \frac{1}{M} \sum_{m=1}^{M} T_m(x)$$

Bootstrap samples



Random Forest

Random Forest adds one more source of diversity:

bootstrap samples $+$ random feature subsets at each split.

At each node, instead of considering all p features, consider only $m < p$.

This decorrelates trees and reduces ensemble variance.

Random Forest algorithm

For $m = 1, \dots, M$:

1. Sample a bootstrap dataset D_m.
2. Grow a tree T_m.
3. At each split, sample a subset of features.
4. Choose the best split only within that subset.

Prediction:

$$\hat{y} = \text{mode}\{T_1(x), \dots, T_M(x)\} \quad \text{or} \quad \hat{y} = \frac{1}{M} \sum_{m=1}^{M} T_m(x).$$

Out-of-bag and feature importance

Because bootstrap samples leave out examples, each tree has out-of-bag observations.

For a given training case, predict it only with trees that did not train on it.

This gives:

- An internal estimate of generalization error.
- A way to compute permutation feature importance.
- A diagnostic for whether more trees still help.

Feature importance is useful, but not causal evidence.

Boosting

Boosting builds an additive model sequentially:

$$F_M(x) = \sum_{m=1}^{M} \alpha_m h_m(x).$$

Each new weak learner focuses on what the current ensemble gets wrong.

Bagging reduces variance by parallel averaging.

Boosting reduces bias by sequential correction.

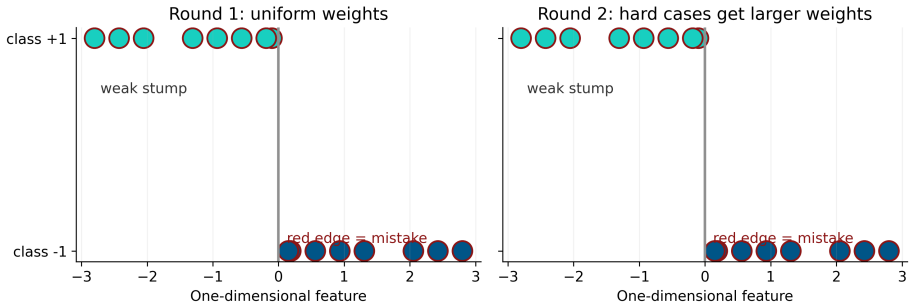
AdaBoost

AdaBoost maintains weights over training examples.

1. Train a weak classifier $h_m(x)$ on weighted data.
2. Increase weights on misclassified examples.
3. Give more vote to classifiers with lower weighted error.

$$\alpha_m = \frac{1}{2} \log \left(\frac{1 - \epsilon_m}{\epsilon_m} \right), \qquad F(x) = \text{sign} \left(\sum_m \alpha_m h_m(x) \right).$$

AdaBoost weights

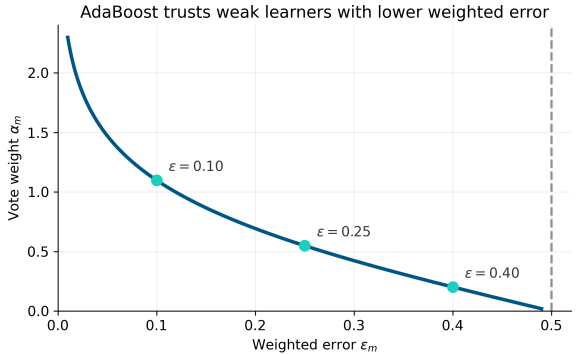


AdaBoost: classifier vote weight

The vote weight is:

$$\alpha_m = \frac{1}{2} \log \left(\frac{1 - \epsilon_m}{\epsilon_m} \right).$$

- Low weighted error $\Rightarrow$ large vote.
- Error near $0.5 \Rightarrow$ almost no vote.
- Error above 0.5 is worse than random guessing.



Gradient boosting

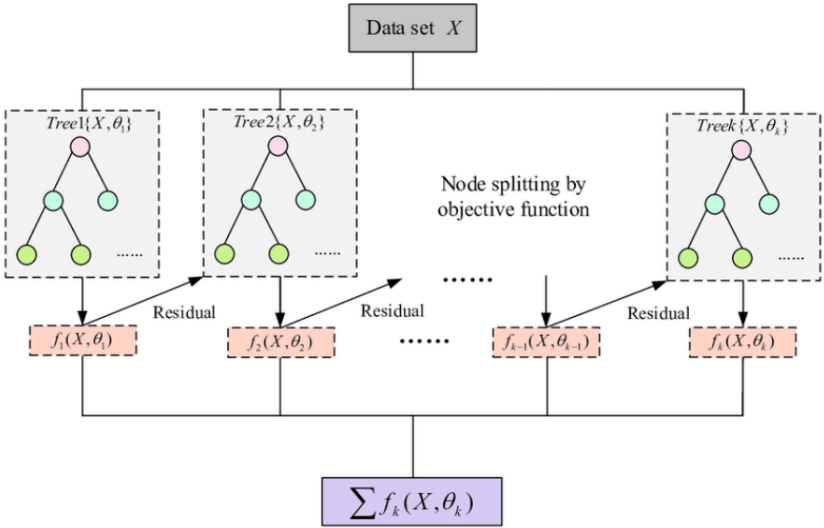
Gradient boosting generalizes boosting to differentiable losses.

At step m, fit a weak learner to the negative gradient:

$$r_i^{(m)} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F=F_{m-1}}.$$

For squared error, these are ordinary residuals.

Gradient boosting



Bagging vs boosting

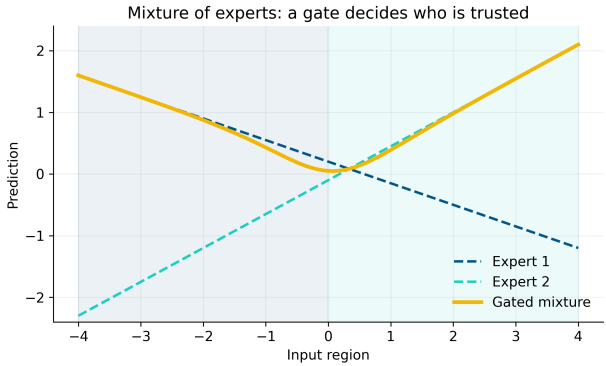
Aspect	Bagging / RF	Boosting
Training	parallel	sequential
Base trees	deep, high variance	shallow, weak learners
Main effect	variance reduction	bias reduction
Diversity	bootstrap/features	reweighting/residuals
Noise sensitivity	moderate	can be high

Both work because they exploit complementary errors.

Mixture of experts

A mixture of experts uses:

- Several specialized models.
- A gating function $g_m(x)$.
- Input-dependent combination weights.



$$F(x) = \sum_{m=1}^{M} g_m(x) f_m(x),$$

$$\sum_{m} g_m(x) = 1.$$

Trees as hard mixtures

A decision tree can be written as:

$$T(x) = \sum_{\ell=1}^{L} \mathbb{1}\{x \in R_\ell\} c_\ell.$$

Each leaf is a simple expert c_ℓ.

The routing function is deterministic and rule-based.

Mixture of experts makes routing soft and learnable.

Practical guidance

For tree ensembles:

- Start with Random Forest for a robust baseline.
- Tune number of trees, maximum depth, leaf size, and feature subsampling.
- Use gradient boosting when performance matters and validation is careful.
- Inspect calibration and class imbalance.
- Do not interpret feature importance as causality.

Trees handle non-linearities and interactions with little preprocessing.

Model calibration

A classifier is calibrated when predicted probabilities match observed frequencies.

If the model says:

$$\hat{p}(y = 1 \mid x) = 0.8$$

for many cases, then roughly 80% of those cases should be positive.

Accuracy and calibration are different questions.

Checking and fixing calibration

Common diagnostics:

- Reliability diagram: predicted probability vs observed frequency.
- Brier score: mean squared error of probabilities.
- Calibration error across probability bins.

Common post-processing methods:

- Platt scaling: fit a logistic calibration model.
- Isotonic regression: fit a monotone non-parametric calibration curve.

Calibrate with held-out or cross-validated predictions.

Feature importance: impurity reduction

During training, every split reduces impurity:

$$\Delta_s = J(P) - \left(\frac{n_L}{n_P} J(L) + \frac{n_R}{n_P} J(R) \right).$$

If split s used feature j, add Δ_s to that feature.

For a forest:

$$I_j = \frac{1}{M} \sum_{m=1}^{M} \sum_{s \in S_{m,j}} \Delta_s.$$

where $S_{m,j}$ are the splits in tree m that use feature j.

A feature is important if it repeatedly makes nodes purer.

Feature importance: permutation

Permutation importance asks a validation question:

1. Measure baseline score, using OOB observations.
2. Randomly shuffle one feature column x_j.
3. Predict again with the trained forest.
4. Importance is the drop in performance.

$$I_j = \text{score}(X, y) - \text{score}(X_{\pi(j)}, y).$$

If shuffling a feature hurts performance, the model was using it.

Missing values in tree ensembles

Missing values create two different questions:

1. Prediction: how should the tree route an example when x_j is missing?
2. Statistics: why is the value missing, and is that missingness informative?

Random Forest is not automatically immune to missing data.

The model or preprocessing pipeline must define a missing-value policy.

Common policies for missing values

Policy	Idea	When useful
Imputation	fill missing values before training	simple baseline / many libraries
Missingness flag	add `is_missing(...)`	missingness itself is predictive
Surrogate split	use another feature when x_j is missing	correlated predictors exist
Default direction	learn left/right path for missing values	tree boosting / native support

Always check what your library actually supports.

Missingness

Trees are more suitable than many linear models because decisions are local:

$$\text{At node } s: \quad \begin{cases} x_j \le t & \to \text{left} \\ x_j > t & \to \text{right} \\ x_j \text{ missing} & \to \text{learned/default path} \end{cases}$$

But missing data can still encode:

- measurement failure;
- selection bias;
- a clinically or operationally meaningful state.

Treat missingness as signal plus uncertainty.

Summary

- Trees learn rules by recursively partitioning feature space.
- Classification trees reduce impurity; regression trees reduce MSE.
- Single trees are interpretable but high variance.
- Bagging and Random Forest reduce variance by averaging diverse trees.
- Boosting builds an additive model that corrects previous errors.
- Diversity and expert specialization are the core ideas.